

Тестирование генераторов случайны чисел (ГСЧ)

Владимир Попов

22 марта 2026 г.

Содержание

1	Общий принцип	3
2	Верификация хи-квадрат	3
2.1	Теоретический хи-квадрат	3
2.2	Оптимизации	5
3	Верификация генератора последовательности случайных битов (ГПСБ)	5
3.1	Теоретическое распределение	5
3.2	Автокорреляция	6
4	Результаты	6

1 Общий принцип

Мы будем рассматривать двухуровневое тестирование. Оно заключается в том, что мы судим о подтверждении гипотезы не из одного набора сэмплов и 1 p-value, а генерируем n набором сэмплов, получаем n p-value и при помощи теста Колмогорова-Смирнова проверяем соответствие полученного набор p-value равномерному распределению.

Для проведения теста Колмогорова-Смирнова нужно посчитать некоторую статистику.

$$\lambda = \sup_{i=\overline{1,n}} |u_i(x) - F(x)| \cdot \sqrt{n} \quad (1)$$

, где n - количество сэмплов, $u_i(x)$ - i -ый сэмпл в точке x , $F(x)$ - теоретическая функция распределения.

По этой статистике уже можно понять (по общепринятым критическим значениям) проходит тест или нет, но также можно получить p-value, чтобы потом можно было опять провести тот же тест по ним. Для этого будем использовать асимптотическое приближение функции распределения Колмагорова.

$$K(\lambda) = 1 - 2 \sum_{k=1}^{\infty} (-1)^{k-1} e^{-2k^2 \lambda^2} \quad (2)$$

$$p\text{-value} = 1 - K(\lambda) = 2 \sum_{k=1}^{\infty} (-1)^{k-1} e^{-2k^2 \lambda^2} \quad (3)$$

Он сходится очень быстро, около 4-5 итераций для точности 10 знаков.

Реализация находится в модуля testing.

2 Верификация хи-квадрат

Проверим правда ли написанный нами генератор соответствует распределению хи-квадрат. Для этого на первом уровне посчитаем p-value при помощи теста Колмогорова-Смирнова, сравнив с теоретическим распределением хи-квадрат. А потом прогоним стандартный второй уровень теста.

2.1 Теоретический хи-квадрат

$$\chi^2(x, n) = \Gamma\left(\frac{n}{2}, \frac{x}{2}\right) = \frac{\gamma(n/2, x/2)}{\Gamma(n/2)} \quad (4)$$

, где n - порядок хи-квадрат распределения, $\Gamma(x, n)$ - Гамма-распределение, $\Gamma(x)$ - Гамма-функция, $\gamma(n, x)$ - нижняя неполная Гамма-функция.

Обе Гамма-функции определяются рекуррентно

$$\begin{aligned}\Gamma(x+1) &= x \cdot \Gamma(x) \stackrel{x \in \mathbb{N}}{=} x! \\ \gamma(x+1, n) &= x \cdot \gamma(x, n) - n^x e^{-n}\end{aligned}\tag{5}$$

Благодаря этому при помощи индукции можно доказать следующие формулы

n - чётное

$$\chi^2(x, n) = 1 - e^{-x/2} \sum_{i=0}^{\frac{n}{2}-1} \frac{(x/2)^i}{i!}\tag{6}$$

n - нечётное

$$\chi^2(x, n) = \operatorname{erf}\left(\sqrt{\frac{x}{2}}\right) - e^{-x/2} \sqrt{\frac{2}{\pi}} \sum_{i=0}^{\frac{n-3}{2}} \frac{(x/2)^{i+0.5}}{\Gamma(i+1.5)}\tag{7}$$

Используя соотношение

$$\Gamma\left(n + \frac{1}{2}\right) = \frac{(2n-1)!!}{2^n} \cdot \sqrt{\pi}\tag{8}$$

Можно преобразовать формулу (7)

$$\chi^2(x, n) = \operatorname{erf}\left(\sqrt{\frac{x}{2}}\right) - e^{-x/2} \cdot \frac{2}{\pi} \cdot \sum_{i=0}^{\frac{n-3}{2}} \frac{x^{i+0.5}}{(2i+1)!!}\tag{9}$$

Как можно видеть в формулах (6) и (9) числитель и знаменатель очень быстро растут и при $i \gtrsim 100$ мы получим переполнение. Чтобы этого избежать будем накапливать в отдельное переменную их отношение.

Выражение для (6)

$$\begin{aligned}\text{term}_0 &= 1 \\ \text{term}_{i+1} &= \text{term}_i \cdot \frac{\frac{x}{2}}{i}\end{aligned}\tag{10}$$

и для (9)

$$\begin{aligned}\text{term}_0 &= \sqrt{x} \\ \text{term}_{i+1} &= \text{term}_i \cdot \frac{x}{2 \cdot i + 1}\end{aligned}\tag{11}$$

Реализация находится в модуле `generators`, файл `ChiSquareGen`

2.2 Оптимизации

Каждый первый уровень считается независимо от другого. Наталкивает на идею распараллелить вычисления. Было решено использовать OpenMP, как самый лёгкий и доступный метод. К циклу расчёта приписываемый прагму

```
1 #pragma omp parallel for schedule(dynamic)
```

Итерации буду динамически распределяться по потокам. Теперь мы не можем использовать один генератор случайных чисел и единственный сид. Поэтому будем конструировать `n_lvl2` сидов (да, это излишнее количество, конечно у нас не так много процессов будет, но это не так уж и долго на фоне асимптотики вложенного цикла) и на каждой итерации инициализировать генератор своим сидом (при этом детерминировано зависящим от изначального заданного пользователем).

3 Верификация генератора последовательности случайных битов (ГПСБ)

Проверим стандартный генератор C++, основный на алгоритме mersenne twister на независимость при помощи теста на автокорреляцию. Для этого на первом уровне будем генерировать последовательность битов длиной `n_lvl1`, считать автокорреляцию с заданным лагом, а после вычислять p-value подставляя значение в функцию биномиального распределения (или нормального, при большом количестве битов).

3.1 Теоретическое распределение

Биномиальное распределение вычисляется по формуле

$$\text{Binom}(x, n, p) = \binom{n}{x} \cdot p^x \cdot (1 - p)^{n-x} = \frac{n! \cdot p^x \cdot (1 - p)^{n-x}}{(n - x)! \cdot x!} \quad (12)$$

Опять же, если мы посчитаем факториалы напрямую, они будут большими, поэтому будем использовать рекуррентное соотношение.

$$\begin{aligned} \text{Binom}(0, n, p) &= (1 - p)^n \\ \text{Binom}(x, n, p) &= \text{Binom}(x - 1, n, p) \cdot \frac{n - x + 1}{x} \cdot \frac{p}{1 - p} \end{aligned} \quad (13)$$

Но при больших n мы все равно не сможем корректно сохранить начальное значение, оно обнулится. Поэтому будем использовать нормальное распределение

$$N(x, \mu, \sigma) = N\left(x = \frac{x - \mu}{\sigma}, 0, 1\right) = \frac{1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)}{2} \quad (14)$$

3.2 Автокорреляция

Нам нужно как-то хранить длинную последовательность бит. В стандартные числа у нас может поместиться только 64 бита. В C++ существует `bitset`, но его размер задаётся константой времени компиляции. Не хотелось бы компилировать всю программу заново, чтобы поменять размер битовых последовательностей. В библиотеке `boost` лежит динамический `bitset`, но мне кажется это слишком тяжёлая зависимость для такой маленькой задачи, поэтому я решил использовать `vector<uint64_t>`, хотя его и неудобно обрабатывать и это будет немного дольше.

Нам нужно сдвинуть битовую последовательность к младшим разрядам на `lag`, а после сделать побитовое исключение или с исходной последовательностью. Количество единиц в получившейся битовой последовательности, за исключение после `lag` битов, и является значением автокорреляции.

Так как мы не можем сдвинуть массив целиком, то для каждого числа будем искать индекс соответствующего ему сдвинутого числа и делать xor. Если `lag` не кратен 64, то нужно будет получить число из 2 соседствующих битовыми операциями.

4 Результаты

Были запущено по 50 тестов с разными седами на каждую конфигурацию (Таблица. 1)

Результаты тестирования можно посмотреть в папке `tests`. В каждом наборе тестов было найдено 1-2 `p-value` значения < 0.01 или > 0.99 . Можно объяснить это случайными выбросами. Все остальные значения получились в рамках нормы, из чего можно судить о корректности генерации чисел хи-квадрат распределения, и о корректности теста автокорреляцией (так как стандартная библиотека уже протестирована, её можно принять за эталон, а после при помощи данной батареи проверять другие ГПСБ).

Таблица 1: Параметры проведения тестов

Тест	Итерации 1-го уровня	Итерации 2-го уровня	Длина последовательности	Параметр теста
Хи-квадрат	10 000	10 000	—	порядок 5
				порядок 50
ГПСБ	—	10 000	1000	lag = 1
				lag = 10
				lag = 200